Report on Homework 2

——Deep Learning

刘滨瑞

2021012579　未央-水木 12

2023 年 12 月 8 日

1 Language modeling with RNN

运行环境为：Windows 10 22H2，Python 3.9.18，PyTorch 1.12.1。

1.1 RNN for Language Modeling

使用 Pytorch 提供的 nn.GRU 类，我实现了基于 GRU 结构的循环神经网络，源代码参见./rnn/model.py 的 49-97 行。

我补全了 train() 函数和 evaluate() 函数，同时完善了模型训练与测试的基本框架，具体实现可以参见./rnn/main.py 中使用注释框出的代码内容。

取训练时的批大小为 20，验证时的批大小为 10；取最大序列长度为 35；RNN 堆叠层数为 2；词嵌入维度数和隐藏层维度数均为 256；使用 Adam 优化器，学习率固定为 0.001；固定随机数种子为 42；调用 train() 函数和 evaluate() 函数，共训练 40 个 epoch，平均训练速度为 2.5min per epoch。

关于模型的训练与验证结果，我们将在下一节中同 LSTM 的实验结果一并展示、分析。

1.2 LSTM Implementation

参考原论文和课件中的相关公式，我还原了 LSTM 中的四个控制门，源代码参见./rnn/model.py 中的 Memory 类。[1] 在此基础上，我仿造 nn.GRU 的接口格式，完成了 LSTM 的整体结构，源代码参见./rnn/model.py 中的 MyLSTM 类。

保持上一节中所使用的超参数不变，调用 train() 函数和 evaluate() 函数，训练 40 个 epoch，平均训练速度为 4.5min per epoch。注意到在相同的超参数下，LSTM 的训练效率明显低于 GRU。

训练过程中，GRU 与 LSTM 在测试集、验证集上 loss 与 perplexity 的变化情况如图 1、图 2 所示。

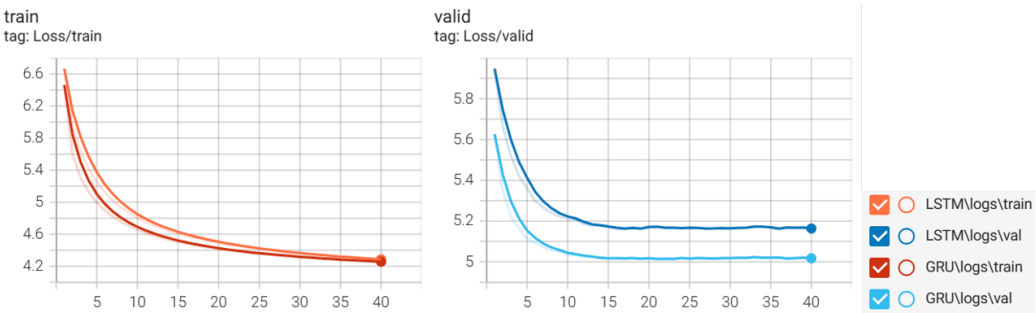

图 1: 训练过程中 GRU 与 LSTM 的 loss 变化情况

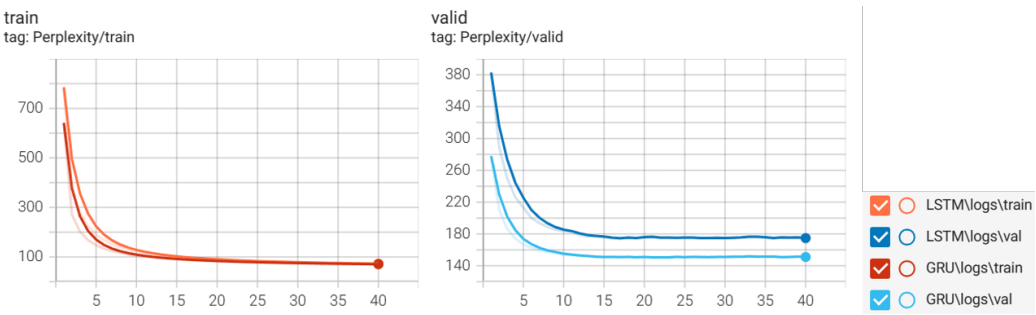

图 2: 训练过程中 GRU 与 LSTM 的 perplexity 变化情况

可以看出，GRU 和 LSTM 的收敛情况良好，在训练集上的 loss 最终均降低至 4.25，perplexity 最终均降低至 70，其中 GRU 的收敛速度略快。

而在验证集上，LSTM 的 loss 最终降至 5.16，perplexity 最终降至 175，而 GRU 的 loss 最终可降至 5.02，perplexity 最终可降至 150。综上，可以认为 GRU 较 LSTM 更容易训练，具有更高的收敛速度，GRU 的泛化能力也要略强于 LSTM。

在训练过程中，我们计算 GRU 与 LSTM 在测试集、验证集上的 Top-1 准确率和 Top-5 准确率，结果如图 3、图 4 所示。

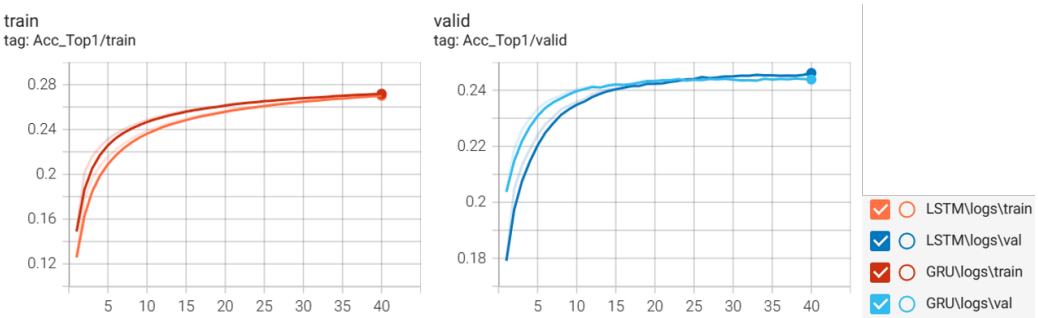

图 3: 训练过程中 GRU 与 LSTM 的 Top-1 准确率变化情况

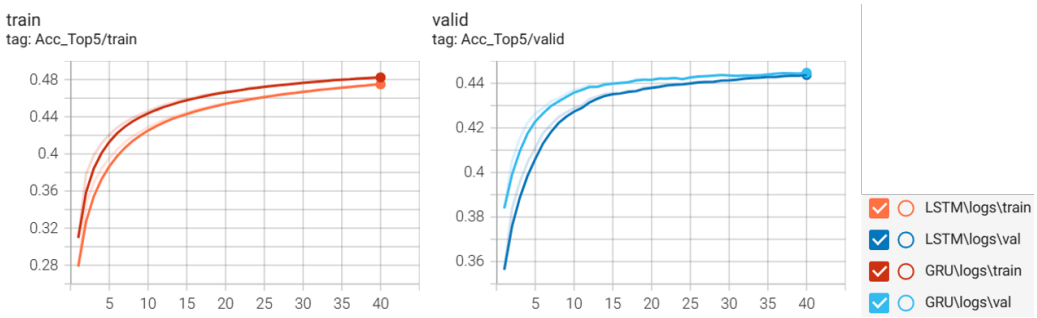

图 4: 训练过程中 GRU 与 LSTM 的 Top-5 准确率变化情况

可以看出，GRU 和 LSTM 的准确率表现几乎一致，我们统计其最佳准确率，展示在表 1 中。

结果显示，在训练集和测试集上，GRU 和 LSTM 的 Top-1 准确率、Top-5 准确率各有高低，但都没有明显的差距。因此，我们倾向与认为 GRU 与 LSTM 在语言学习任务中的性能是等同的。这证明了相对于 LSTM，GRU 所采用的简化结构的合理性与有效性。

表 1: GRU 和 LSTM 的最佳准确率结果

model	Top-1 Accuracy	Top-5 Accuracy
GRU (train)	27.20%	48.27%
LSTM (train)	27.04%	47.53%
GRU (validation)	24.37%	44.49%
LSTM (validation)	24.66%	44.39%

1.3 Inference using RNN

我补全了 predict() 函数，源代码参见./rnn/main.py。

predict() 函数使用了 Beam Search 和 Auto-Regressive 进行语言推理。其实现过程可分为如下若干步：

1. 首先，我们需要载入完成训练的 RNN，这里使用的是 GRU。然后，我们载入 data_loader 提供的词表和词典，用于翻译。

2. 我们先将待补全的句子 token 化，然后将其输入 RNN 中。RNN 会输出一个 Tensor，该 Tensor 的第一维代表着 sequence。取该维度上的最后一位上的 Tensor 切片，即为模型给出的下一个词语的概率分布的预测。

3. 基于此预测，我们即可以判断哪些词更有可能被填入句子的下一个位置。而在 Beam Search 中，我们总是保留前 K 个最有可能的词语选项。

4. 对于 K 个词语中的每一个，我们都将其填入句子的末尾，视为已知，然后重新将整个句子输入 RNN。重复 Beam Search 步骤，我们将得到 K^2 个最有可能的词语选项组合，我们仅保留其中 K 个联合概率最大的。

5. 重复第 4 步，直到词语序列达到指定长度。

值得一提的是，推理过程中的预测比例可以通过修改输入文本长度与指定序列长度的值调整。另外，Beam Search 的搜索宽度 K 也是一个重要的参数，直接影响着推理的复杂度与效果。

使用 predict() 函数，我们输入部分自定义文本和验证集中的文本，检查 RNN 的自回归推理结果。这部分的代码参见./rnn/inference.py 文件。

首先是部分自定义文本的输入。取 $K = 10$，我们输入了以下三句文本，要求补充 8 个词语，结果如图 5 所示：

1. I am a student . I love

2. I am a student . I live in America . I love

3. I am a student . I live in America . I do not love money . I love

```
size of train set:  2088628
size of valid set:  217646
*****************************************************************************************
pred 1 :
I am a student . I love a young woman who was a young woman
pred 2 :
I am a student . I love a young woman who is a young woman
pred 3 :
I am a student . I love a young woman who was a young man
*****************************************************************************************
pred 1 :
I am a student . I live in America . I love a lot of money . " <eos> <eos>
pred 2 :
I am a student . I live in America . I love a lot of money . " <eos> Walpole
pred 3 :
I am a student . I live in America . I love a lot of money . " <eos> In
*****************************************************************************************
pred 1 :
I am a student . I live in America . I do not love money . I love the song . " <eos> <eos> = =
pred 2 :
I am a student . I live in America . I do not love money . I love the song . " <eos> <eos> <eos> =
pred 3 :
I am a student . I live in America . I do not love money . I love the song . " <eos> O 'Malley 's
```

图 5: 自定义文本的自回归推理结果

可以看出，模型能够理解输入语句的含义，并给出符合英文语法和简单逻辑的补全。

然后，我们从验证集摘取 10 段语句，这 10 段语句的原文与摘取位置详见./rnn/inference.py 文件。去掉这 10 段语句尾部的若干个词语，将它们

输入模型，令其进行推理预测，结果如图 6 所示。

```
**********************************************************************************************
pred 1 :
The city 's growth has reflected the push and pull of many social and cultural activities . <eos> <eos> = = =
**********************************************************************************************
pred 1 :
Lewis McAllister , a businessman in Tuscaloosa , Alabama , was the first Republican to serve in the Mississipp
i House of San José del Toro . <eos> <eos> =
**********************************************************************************************
pred 1 :
The secrecy surrounding the circumstance of his death caused speculation for the Augustan History . <eos> <eos
> = =
**********************************************************************************************
pred 1 :
The film was released on 3 February 2012 in the United States and Canada , and was released on October 3 , 201
2 . <eos> <eos>
**********************************************************************************************
pred 1 :
That court , noting Hubbard 's instruction that Scientologists should make money , make more money – make othe
r people produce so as to <unk> . <eos> <eos> = = = =
**********************************************************************************************
pred 1 :
Given the history of Nazism 's rise to power in Germany in the 1930s , the present German state has committed
itself to taking active steps to prevent the rise of any ideology that threatens the values enshrined in <unk>
 . <eos> <eos> = = = <unk>
**********************************************************************************************
pred 1 :
Sonic the Hedgehog is a platform video game developed by Sonic Team and published by Sega for Crash Boom Boom
Boom Bang / Bang !
**********************************************************************************************
pred 1 :
During the tour , several bits and performances were filmed and later edited into a VHS and DVD entitled Aroun
d <unk> . <eos> <eos> = = Track listing
**********************************************************************************************
pred 1 :
Australia entered the Second World War on 27 June . <eos> <eos> = = =
**********************************************************************************************
pred 1 :
Fort Scott National Historic Site is a historical area under the control of the United States National Park Se
rvice in Bourbon County , Kansas , New Jersey . <eos> <eos> = = =
```

图 6: 部分验证集文本的自回归推理结果

可以看出，预测结果有好有坏。第一句、第九句和第十句的补全效果较好，大致符合语法和逻辑，内容也与原文相近。而第二句、第五句和第八句的补全效果较差，有点不知所云。

我认为，模型补全部分语句的效果较差，一方面是因为训练集规模过小，有一些知识性的内容——如 Sonic 是哪家公司制作的——不能够学习到；另一方面则是因为 RNN 对于自然语言特征的提取能力仍然有限，尚且不能够理解一些复杂的语法与逻辑。

1.4 Question Answering

使用字母级别的 RNN 的优势：

我们注意到，在字母级别，RNN 需要处理的仅仅是包含几十个字符的字母表，而非规模以万计的词汇表，这使得 RNN 中输入层和输出层的维度数可以大幅减小。另外，字母作词嵌入后的大小远小于单词作词嵌入后的大小。这都将使得模型的参数量和复杂度显著降低，同时也将允许模型能够处理更大的词汇量规模的任务。

使用字母级别的 RNN 的劣势：

在字母级别，RNN 需要从字母层面理解自然语言，这一任务的难度非常高，这要求 RNN 在学习如何组织单词之前，还需要先学会如何拼写单词、理解单词。这一方面使得 RNN 需要更大规模的数据进行训练，提高了训练难度，另一方面也减小了 RNN 输出可被人类理解的文本的可能。

2 Graph Neural Networks and Node2Vec

运行环境为：Windows 10 22H2，Python 3.9.18，PyTorch 1.12.1，Torch Geometric 2.0.3。

2.1 Task A and Task B

参考原论文，基于 Pytorch Geometric 库提供的 MLP、GINConv 等基本结构模块，我重新实现了 GIN。[2] 为了让 GIN 具备更好的性能，我调整了 dropout 层和 relu 函数在网络中的位置，同时适当地调整了网络层数、隐藏层维度等超参数。源代码参见./gnn/gin.py。

运行./gnn/gcn.py、./gnn/gat.py、./gnn/node2vec.py 和./gnn/gin.py，记录模型在训练过程中的损失值变化情况，记录模型在训练集、验证集和测试集上的准确率变化情况，结果如图 7，图 8 所示。

可以看出，各模型的收敛情况良好。Node2Vec 的收敛速度最快，GIN 次之，GAT 的收敛速度最慢。

GAT、GAN 和 GIN 都能够完全拟合训练集，最终在训练集上达到接近 100% 的准确率。在验证集上，GAT、GAN 的准确率可达 80%，GIN 的

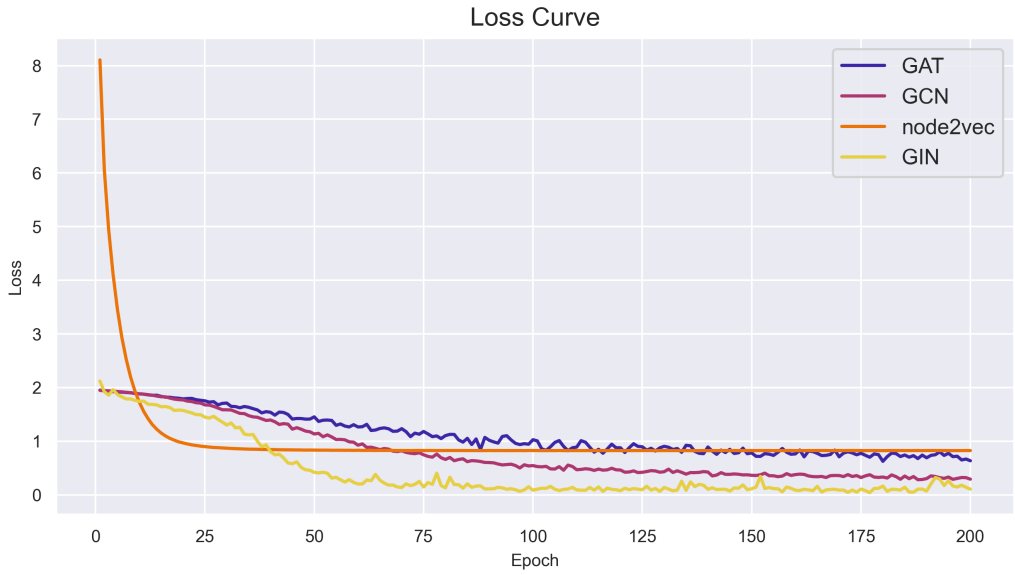

图 7: 在训练过程中，各 GNN 的 loss 变化情况

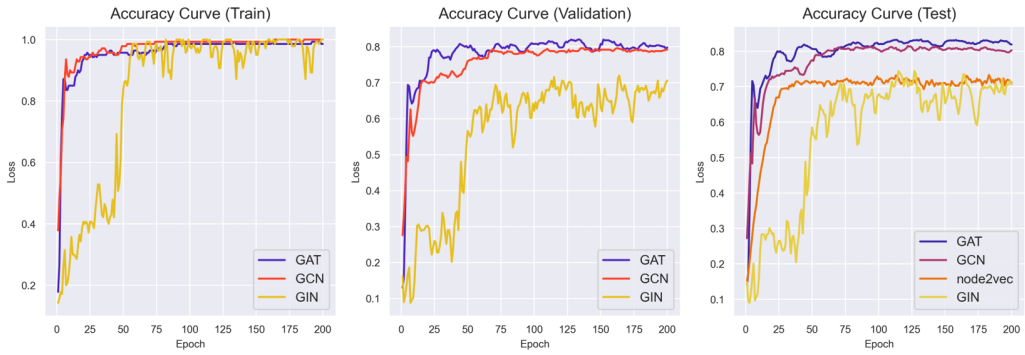

图 8: 在训练过程中，各 GNN 的准确率变化情况

准确率可达 70%。在测试集上，各模型的表现同验证集类似，GAT、GAN 的最终准确率为 80%，Node2Vec、GIN 的最终准确率为 70%。

如图 9 所示，使用训练后的 Node2Vec 模型生成 TSNE 图。从图中可以直观地看出，Node2Vec 对图数据集的分类效果良好。我们有理由相信，性能较 Node2Vec 更好的 GAT、GAN 等 GNN 模型同样可以较为成功地完成对图数据集的分类任务。

图 9: Node2Vec 模型生成的 TSNE 图

Notes

[1]Hochreiter, S., and Schmidhuber, J. (1997). Long short-term memory. Neural computation.

[2]Xu, K., Li, C., Tian, Y., Sonobe, T., Kawarabayashi, K., and Jegelka, S. (2018). How powerful are graph neural networks? In Proceedings of the International Conference on Learning Representations (ICLR).